

CHUQUR O'RGANISH USULLARI · Ma'ruza konspekti (Lecture Note)

2-ma'ruza. Backpropagation va optimizatorlar

2026–2027 o'quv yili

Reja. 1) Hisoblash grafi va zanjir qoidasi. 2) Softmax va cross-entropy. 3) Backpropagation ning matritsa shakli. 4) Qo'lda hisoblangan misollar. 5) Murakkablik, xotira, gradient tekshiruvi. 6) Gradientning so'nishi va portlashi. 7) Optimizatorlar: SGD, Momentum, Nesterov, RMSProp, Adam, AdamW; qadam jadvallari. 8) Python (NumPy, PyTorch). 9) Nazorat savollari va mashqlar.

O'quv natijalari. Ma'ruzadan so'ng talaba: ko'p qatlamli tarmoq uchun barcha gradientlarni matritsa shaklida chiqaradi va o'lchamlarini tekshiradi; softmax–cross-entropy gradientini isbotlaydi; oldinga/orqaga yurishni qo'lda bajaradi; gradientni sonli usul va autograd bilan tekshiradi; optimizatorlarning yangilash qoidalarini, Adam siljish tuzatmasini asoslaydi va qadam uzunligini tanlaydi.

Tayanch iboralar. hisoblash grafi, zanjir qoidasi, Yakobi matritsasi, backpropagation (orqaga tarqalish), logit, softmax, cross-entropy, xato signali δ , gradient tekshiruvi, vanishing/exploding gradient, gradient clipping, SGD, Momentum, Nesterov, RMSProp, Adam, AdamW, learning rate (qadam uzunligi), warmup, cosine schedule.

1. Muammo: P ta hosilani qanday tez topish mumkin?

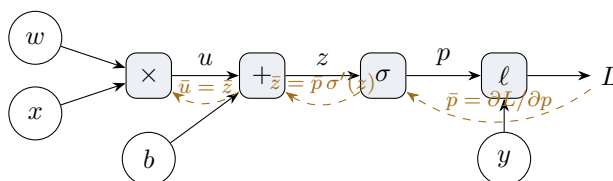
O'qitish uchun yo'qotish $L(\theta)$ ning barcha P ta parametr bo'yicha gradienti kerak. Eng sodda yo'l — *sonli differensiallash*:

$$\frac{\partial L}{\partial \theta_p} \approx \frac{L(\theta + \varepsilon \mathbf{e}_p) - L(\theta - \varepsilon \mathbf{e}_p)}{2\varepsilon}.$$

U to'g'ri, lekin har bir hosila uchun ikkita to'liq oldinga yurish kerak: jami $2P$ ta. ResNet-50 da $P \approx 2,6 \cdot 10^7$ — bitta qadam uchun 50 milliondan ortiq oldinga yurish. *Backpropagation* (orqaga tarqalish, Rumelhart–Hinton–Williams, 1986) esa **bitta oldinga va bitta orqaga** yurish bilan barcha P ta hosilani topadi; narxi oldinga yurishning taxminan 3 barobaridan oshmaydi.

1.1. Hisoblash grafi va zanjir qoidasi

Har qanday tarmoq — elementar amallardan tuzilgan *yo'naltirilgan asiklik graf*. Masalan, $L = \ell(\sigma(wx + b), y)$:



Har bir tugun uchun $\bar{v} := \partial L / \partial v$ (“adjoint”) belgilanadi. Agar v tugun $u_1, \dots, u_r$ tugunlarga kirs, ko'p o'zgaruvchili zanjir qoidasi bo'yicha

$$\bar{v} = \sum_{j=1}^r \bar{u}_j \frac{\partial u_j}{\partial v}.$$

Tugunlarni teskari topologik tartibda ko'rib chiqsak, har bir $\bar{v}$ bir marta hisoblanadi — bu *reverse-mode* avtomatik differensiallash (PyTorch **autograd**) va backpropagation ning mohiyati. Vektor ko'rinishida: $\mathbf{u} = f(\mathbf{v})$, $\mathbf{J} = \partial \mathbf{u} / \partial \mathbf{v}$ — Yakobi matritsasi bo'lsa, $\bar{\mathbf{v}} = \mathbf{J}^T \bar{\mathbf{u}}$ (vektor–Yakobi ko'paytmasi); $\mathbf{J}$ ning o'zini tuzish shart emas.

Izoh 1.1 (Forward va reverse mode). Forward-mode bitta *kirish* yo'nalishi bo'yicha hosilani bir o'tishda beradi (Yakobi–vektor ko'paytmasi); P ta parametr uchun P ta o'tish kerak. Reverse-mode bitta *chiqish* bo'yicha barcha hosilalarni bir o'tishda beradi. Neyron tarmoqda chiqish bitta (skalyar L), parametrlar millionlab — shuning uchun reverse-mode tanlanadi.

Misol 1.2 (Grafda raqamli orqaga yurish). Yuqoridagi grafda $w = 2$, $x = 1$, $b = -1$, $y = 1$, $\ell = -y \ln p - (1 - y) \ln(1 - p)$.

Yechim. Oldinga: $u = 2$, $z = 1$, $p = \sigma(1) \approx 0,7311$, $L = -\ln 0,7311 \approx 0,3133$. Orqaga: $\bar{p} = -1/p \approx -1,3679$; $\bar{z} = \bar{p}p(1-p) \approx -1,3679 \cdot 0,1966 = -0,2689$ ($= p - y$, 1-ma'ruzaga mos); $\bar{u} = \bar{b} = \bar{z} = -0,2689$; $\bar{w} = \bar{u}x = -0,2689$, $\bar{x} = \bar{u}w \approx -0,5379$. Har bir tugunda faqat *mahalliy* hosila ishlatildi. $\triangleleft$

Amal (oldinga)	Mahalliy orqaga qoida	Izoh
$c = a + b$	$\bar{a} += \bar{c}, \bar{b} += \bar{c}$	gradientni "tarqatadi"
$c = ab$	$\bar{a} += \bar{c}b, \bar{b} += \bar{c}a$	"almashtiradi"
$C = AB$	$\bar{A} += \bar{C}B^T, \bar{B} += A^T\bar{C}$	o'lchamlar bo'yicha tekshiriladi
$\mathbf{c} = \mathbf{a} + b\mathbf{1}$ (broadcast)	$\bar{b} += \mathbf{1}^T\bar{\mathbf{c}}$	broadcast $\leftrightarrow$ yig'indi
$c = \max(a, b)$	$\bar{c}$ faqat katta argumentga	ReLU, max-pooling
$\mathbf{c} = \varphi(\mathbf{a})$	$\bar{\mathbf{a}} += \bar{\mathbf{c}} \odot \varphi'(\mathbf{a})$	elementlar bo'yicha

Bir tugun bir necha joyda ishlatilsa, gradientlar *qo'shiladi* ($+=$) — PyTorch'da `.grad` ning to'planishi va `zero_grad()` zarurati shundan.

2. Belgilar va tarmoq tuzilishi

L qatlamli tarmoq, paket $A^{(0)} = X \in \mathbb{R}^{n \times d_0}$ (qatorlar — namunalar). $i = 1, \dots, L$:

$$Z^{(i)} = A^{(i-1)}W^{(i)} + \mathbf{1}\mathbf{b}^{(i)T}, \quad A^{(i)} = \varphi(Z^{(i)}) \quad (i < L), \quad Z^{(L)} \text{ — logitlar.}$$

Belgi	O'lchami	Ma'nosi
$A^{(i-1)}$	$n \times d_{i-1}$	i -qatlamga kirish
$W^{(i)}, \mathbf{b}^{(i)}$	$d_{i-1} \times d_i, d_i$	og'irliklar, siljish (har bir qatorga qo'shiladi)
$Z^{(i)}$	$n \times d_i$	oldindan faollashtirish
$\delta^{(i)} = \partial L / \partial Z^{(i)}$	$n \times d_i$	i -qatlamdagi xato signali

Qoida: ∇W ning o'lchami har doim W niki bilan bir xil. Formulalarda har bir ko'paytuvchi ostiga o'lchamini yozib borish ko'p xatolarning oldini oladi.

3. Softmax va cross-entropy

Ta'rif 3.1. $\mathbf{z} \in \mathbb{R}^K$ logitlar uchun $\text{softmax}(\mathbf{z})_j = p_j = \frac{e^{z_j}}{\sum_{c=1}^K e^{z_c}}$. To'g'ri sinf y bo'lsa, *cross-entropy* yo'qotish $\ell = -\ln p_y$; paket bo'yicha $L = -\frac{1}{n} \sum_k \ln p_{k,y_k}$. One-hot yorliq $\mathbf{y}$ bilan $\ell = -\sum_j y_j \ln p_j$.

Xossa 3.2. (i) $p_j > 0$, $\sum_j p_j = 1$; (ii) $\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z})$; (iii) $\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)$, ya'ni $J = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^T$ (simmetrik, $J\mathbf{1} = \mathbf{0}$).

Isbot. (ii) surat va maxraj e^c ga ko'payadi. (iii) $p_i = e^{z_i}/S$, $S = \sum_c e^{z_c}$: $\partial p_i / \partial z_j = (\delta_{ij} e^{z_i} S - e^{z_i} e^{z_j}) / S^2 = p_i \delta_{ij} - p_i p_j$. $J\mathbf{1}$ ning i -komponenti $p_i - p_i \sum_j p_j = 0$. $\square$

(ii) amaliy jihatdan muhim: e^{1000} toshib ketadi, shuning uchun hisobda $\mathbf{z} - \max_j z_j$ olinadi va $\ln p_y = z_y - m - \ln \sum_c e^{z_c - m}$ (log-sum-exp hiylasi).

Teorema 3.3 (Softmax-cross-entropy gradienti). $\frac{\partial \ell}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$; paket uchun $\delta^{(L)} = \frac{\partial L}{\partial Z^{(L)}} = \frac{1}{n} (P - Y)$, bunda P — softmax ehtimolliklari, Y — one-hot matritsa.

Isbot. $\ell = -z_y + \ln \sum_c e^{z_c}$, demak $\partial \ell / \partial z_j = -\mathbf{1}[j = y] + e^{z_j} / \sum_c e^{z_c} = p_j - y_j$. Boshqacha: $\partial \ell / \partial \mathbf{z} = J^T(-\mathbf{y}/\mathbf{p})$; j -komponent $-\sum_i y_i (\delta_{ij} - p_j) = -y_j + p_j$. L o'rtacha bo'lgani uchun $1/n$ ko'paytuvchi. $\square$

Gradient yana "xatoning o'zi": $p_y = 0,7$ bo'lsa, to'g'ri sinf logiti $-0,3$ gradient oladi (ya'ni oshiriladi), qolgan sinflar logitlari p_j gradient bilan kamaytiriladi. $K = 2$ da bu 1-ma'ruzadagi sigmoid-binary cross-entropy natijasiga aylanadi.

Amaliy qoida

PyTorch'da `nn.CrossEntropyLoss` logitlarni qabul qiladi va softmax ni ichida (log-sum-exp bilan) bajaradi. Model oxiriga alohida `Softmax` qo'yilsa, softmax ikki marta qo'llanadi: gradientlar kichrayadi va o'qitish buziladi.

4. Backpropagation ning matritsa shakli

Matritsalar uchun $\langle A, B \rangle = \text{tr}(A^T B)$ skalyar ko'paytma va differensial $dL = \langle \nabla_M L, dM \rangle$ dan foydalanamiz.

Teorema 4.1 (Backpropagation). $\delta^{(i)} = \partial L / \partial Z^{(i)}$ ma'lum bo'lsa,

$$\nabla W^{(i)} = A^{(i-1)T} \delta^{(i)}, \quad \nabla \mathbf{b}^{(i)} = \delta^{(i)T} \mathbf{1}, \quad \delta^{(i-1)} = (\delta^{(i)} W^{(i)T}) \odot \varphi'(Z^{(i-1)})$$

Rekursiya $\delta^{(L)} = \frac{1}{n}(P - Y)$ dan boshlanadi (Teorema 3.3).

Isbot. $Z = AW + \mathbf{1}b^T$ dan $dZ = dA W + A dW + \mathbf{1} db^T$. Demak

$$dL = \langle \delta, dZ \rangle = \langle \delta W^T, dA \rangle + \langle A^T \delta, dW \rangle + \langle \delta^T \mathbf{1}, db \rangle,$$

bunda $\text{tr}(\delta^T A dW) = \text{tr}((A^T \delta)^T dW)$ va $\text{tr}(\delta^T dA W) = \text{tr}((\delta W^T)^T dA)$ (izning siklik xossasi). Bu ∇W , $\nabla \mathbf{b}$ va $\nabla A^{(i-1)} = \delta^{(i)} W^{(i)T}$ ni beradi. $A^{(i-1)} = \varphi(Z^{(i-1)})$ elementlar bo'yicha: $dA = \varphi'(Z) \odot dZ$, va $\langle G, \varphi' \odot dZ \rangle = \langle G \odot \varphi', dZ \rangle$, demak $\delta^{(i-1)} = \nabla A^{(i-1)} \odot \varphi'(Z^{(i-1)})$. $\square$

O'lchamlar: $(d_{i-1} \times n)(n \times d_i) = d_{i-1} \times d_i$; $(n \times d_i)(d_i \times d_{i-1}) = n \times d_{i-1}$ — $\varphi'(Z^{(i-1)})$ bilan mos. Talqin: xato signali og'irliklar orqali *orqaga tarqaladi* (W^T), faol bo'lmagan neyron ($\varphi' = 0$) esa hech qanday signal o'tkazmaydi.

Algoritmi 1. Backpropagation (bitta paket)

1. *Oldinga:* $A^{(0)} = X$; $i = 1, \dots, L$: $Z^{(i)} = A^{(i-1)}W^{(i)} + \mathbf{1}b^{(i)T}$, $A^{(i)} = \varphi(Z^{(i)})$ ($i < L$). Barcha $A^{(i)}, Z^{(i)}$ **saqlanadi**.
2. *Yo'qotish:* $P = \text{softmax}(Z^{(L)})$ (qatorlar bo'yicha), $L = -\frac{1}{n} \sum_k \ln P_{k,y_k}$.
3. *Orqaga:* $\delta = \frac{1}{n}(P - Y)$; $i = L, \dots, 1$: $\nabla W^{(i)} = A^{(i-1)T} \delta$, $\nabla \mathbf{b}^{(i)} = \delta^T \mathbf{1}$; agar $i > 1$: $\delta \leftarrow (\delta W^{(i)T}) \odot \varphi'(Z^{(i-1)})$.
4. *Yangilash:* optimizator qoidasi bilan (8-bo'lim).

5. Qo'lda hisoblangan misollar

Misol 5.1 (ReLU, ikki qatlam). $n = 1$, $\mathbf{x} = (1, 2)$, $W^{(1)} = I_2$, $\mathbf{b}^{(1)} = (0, -3)$, $W^{(2)} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, $\mathbf{b}^{(2)} = \mathbf{0}$, ReLU, to'g'ri sinf — birinchi ($\mathbf{y} = (1, 0)$).

Yechim. *Oldinga.* $Z^{(1)} = (1, -1)$, $A^{(1)} = (1, 0)$, $Z^{(2)} = (1, 1)$, $\mathbf{p} = (0, 5; 0, 5)$, $L = \ln 2 \approx 0,6931$. *Orqaga.* $\delta^{(2)} = \mathbf{p} - \mathbf{y} = (-0,5; 0,5)$,

$$\nabla W^{(2)} = A^{(1)T} \delta^{(2)} = \begin{pmatrix} -0,5 & 0,5 \\ 0 & 0 \end{pmatrix}, \quad \nabla \mathbf{b}^{(2)} = (-0,5; 0,5), \quad \delta^{(2)} W^{(2)T} = (0; -1), \quad \varphi'(Z^{(1)}) = (1, 0).$$

$\delta^{(1)} = (0; -1) \odot (1; 0) = (0; 0)$, demak $\nabla W^{(1)} = 0$, $\nabla \mathbf{b}^{(1)} = \mathbf{0}$. Ikkinchi yashirin neyron o'chiq ($z = -1$): u signal o'tkazmaydi. Birinchi neyron *faol*, lekin unga kelgan signal $(-0,5) \cdot 1 + 0,5 \cdot 1 = 0$ — $\delta^{(2)}$ ning $W^{(2)}$ birinchi qatoriga ortogonaligi natijasi (ikkala logitga bir xil hissa qo'shadi). Ya'ni birinchi qatlam gradienti nol bo'lishining ikki xil sababi bor: o'chiq neyron va tasodifiy ortogonallik. $\triangleleft$

Misol 5.2 (Xuddi shu tarmoq, tanh).

Yechim. $Z^{(1)} = (1, -1)$, $A^{(1)} = (\tanh 1, -\tanh 1) = (0,7616; -0,7616)$, $Z^{(2)} = (0; 1,5232)$, $\mathbf{p} = (0,1790; 0,8210)$, $L = -\ln 0,1790 \approx 1,7204$. $\delta^{(2)} = (-0,8210; 0,8210)$, $\nabla W^{(2)} = \begin{pmatrix} -0,6253 & 0,6253 \\ 0,6253 & -0,6253 \end{pmatrix}$. $\delta^{(2)}W^{(2)T} = (0; -1,6420)$, $\tanh'(\pm 1) = 1 - 0,7616^2 = 0,4200$, $\delta^{(1)} = (0; -0,6896)$, $\nabla W^{(1)} = \mathbf{x}^T \delta^{(1)} = \begin{pmatrix} 0 & -0,6896 \\ 0 & -1,3792 \end{pmatrix}$, $\nabla \mathbf{b}^{(1)} = (0; -0,6896)$. Endi ikkinchi yashirin neyron ham o'rganadi, chunki $\tanh' > 0$ hamma joyda. (Barcha sonlar PyTorch autograd bilan tekshirilgan.) $\triangleleft$

6. Murakkablik, xotira va gradient tekshiruvi

Hisoblash. To'liq bog'langan qatlam oldinga yurishda $Z = AW$ uchun $\approx 2nd_{i-1}d_i$ FLOP sarflaydi; orqaga yurishda ikkita shunday ko'paytma ($A^T\delta$ va δW^T) — $\approx 4nd_{i-1}d_i$. Demak bitta o'qitish qadami oldinga yurishdan ≈ 3 marta qimmat, sonli usul esa $2P$ marta.

Xotira. Orqaga yurish uchun barcha $A^{(i-1)}$ va $Z^{(i-1)}$ (yoki φ') saqlanadi: xotira $\sim n \sum_i d_i$ — paket hajmi va chuqurlikka proporsional. Shuning uchun o'qitish xotirasi model faylidan ancha katta va katta paket “out of memory” beradi. Yechimlar: kichikroq paket, gradient accumulation, activation checkpointing (qayta hisoblash evaziga xotira tejash), aralash aniqlik (mixed precision).

6.1. Gradient tekshiruvi

Analitik gradient $\mathbf{g}_a$ sonli $\mathbf{g}_s$ bilan nisbiy xato orqali solishtiriladi: $\frac{\|\mathbf{g}_a - \mathbf{g}_s\|_\infty}{\max(\|\mathbf{g}_s\|_\infty, \epsilon_0)}$. float64 da to'g'ri kod uchun odatda 10^{-7} dan kichik (bizning tajribada $4,9 \cdot 10^{-10}$).

Xossa 6.1 (Markaziy ayirma xatosi). $f \in C^3$ bo'lsa, $\frac{f(t+\varepsilon) - f(t-\varepsilon)}{2\varepsilon} = f'(t) + \frac{\varepsilon^2}{6}f'''(\xi)$. Yaxlitlash xatosi (mashina aniqligi $u \approx 1,1 \cdot 10^{-16}$) $\approx u|f|/\varepsilon$ ga teng, shuning uchun umumiy xato $\approx \frac{\varepsilon^2}{6}|f'''| + \frac{u|f|}{\varepsilon}$ va optimal $\varepsilon \sim u^{1/3} \approx 5 \cdot 10^{-6}$.

Isbot. Teylor: $f(t \pm \varepsilon) = f \pm \varepsilon f' + \frac{\varepsilon^2}{2}f'' \pm \frac{\varepsilon^3}{6}f'''(\xi_{\pm})$; ayirib 2ε ga bo'lsak, juft hadlar qisqaradi; o'rta qiymat teoremasi bilan $\frac{1}{2}(f'''(\xi_+) + f'''(\xi_-)) = f'''(\xi)$. Minimum: $\frac{d}{d\varepsilon}(\frac{\varepsilon^2}{6}a + \frac{ub}{\varepsilon}) = 0 \Rightarrow \varepsilon^3 = 3ub/a$. $\square$

Demak $\varepsilon = 10^{-2}$ da kesish xatosi, $\varepsilon = 10^{-12}$ da yaxlitlash xatosi katta. ReLU ($z = 0$ da sinish) bilan tekshiruv, agar biror $|z| < \varepsilon$ bo'lsa, formula to'g'ri bo'lsa ham katta xato beradi: tekshiruvni silliq faollashtirish (tanh) bilan o'tkazish kerak.

7. Gradientning so'nishi va portlashi

Teorema 4.1 rekursiyasini ochsak, $D^{(i)} = \varphi'(Z^{(i)})$ bilan

$$\delta^{(1)} = \left(\dots ((\delta^{(L)}W^{(L)T}) \odot D^{(L-1)})W^{(L-1)T} \dots \right) \odot D^{(1)}.$$

Bitta namuna uchun

$$\|\delta^{(1)}\| \leq \|\delta^{(L)}\| \prod_{i=2}^L \|W^{(i)}\|_2 \max |\varphi'|.$$

Sigmoid ($\max \sigma' = \frac{1}{4}$) va $\|W\|_2 \leq 1$ bo'lsa, $\|\delta^{(1)}\| \leq 4^{-(L-1)}\|\delta^{(L)}\|$ — *vanishing gradient*: birinchi qatlamlar deyarli o'rganmaydi. Aksincha, $\|W\|_2$ katta bo'lsa, ko'paytma eksponensial o'sadi — *exploding gradient* (nan). Qarshi choralar: ReLU oilasi, He/Glorot boshlang'ich qiymatlari, batch normalization, residual bog'lanishlar (3-ma'ruza) va *gradient clipping*: $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$ (clip_grad_norm_).

7.1. Dropout va mini-batch shovqini

Dropout (Srivastava va boshq., 2014): o'qitishda har bir faollashtirish q ehtimollik bilan saqlanadi, $m_j \sim \text{Bernoulli}(q)$, $\tilde{a}_j = m_j a_j / q$ (“inverted dropout”); $\mathbb{E}[\tilde{a}_j] = a_j$, shuning uchun inferensiyada dropout shunchaki o'chiriladi (model.eval()). Orqaga yurishda xuddi shu niqob qo'llanadi: $\tilde{a}_j = m_j \tilde{a}_j / q$. Dropout neyronlarning o'zaro moslashib qolishiga to'sqinlik qiladi va ko'p modellar ansambliga o'xshash regularizatsiya beradi. PyTorch'da nn.Dropout(p) parametri $p = 1 - q$ — o'chirish ehtimolligi.

Mini-batch shovqini. Paket gradienti $\mathbf{g}_B = \frac{1}{B} \sum_{k \in B} \nabla \ell_k$ to'liq gradientning siljimagan bahosi; namunalar tasodifiy (qaytarib) tanlansa, $\text{Cov}(\mathbf{g}_B) = \frac{1}{B} \Sigma$, Σ — bitta namuna gradienti kovariatsiyasi. Demak shovqinning standart chetlanishi $\propto 1/\sqrt{B}$: B ni 4 marta oshirish shovqinni faqat 2 marta kamaytiradi. Amaliy evristika (“linear scaling rule”, Goyal va boshq., 2017): B k marta oshirilsa, η ham taxminan k marta oshiriladi (warmup bilan). Kichik paket shovqini ba'zan foydali — keskin minimumlardan chiqishga yordam beradi.

8. Optimizatorlar

Gradient faqat yo'nalishni beradi; qadam qanday tashlanishi — optimizatorning ishi. $\mathbf{g}_t = \nabla L_{\mathcal{B}_t}(\theta_{t-1})$ — t -paketdagi gradient.

Usul	Yangilash qoidasi	G'oya
SGD	$\theta_t = \theta_{t-1} - \eta \mathbf{g}_t$	eng sodda
Momentum	$\mathbf{v}_t = \mu \mathbf{v}_{t-1} + \mathbf{g}_t$, $\theta_t = \theta_{t-1} - \eta \mathbf{v}_t$	gradientlar tarixi (inersiya)
Nesterov	$\mathbf{g}_t$ ni $\theta_{t-1} - \eta \mu \mathbf{v}_{t-1}$ nuqtada hisoblash	“oldinga qarash”
RMSProp	$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1 - \rho) \mathbf{g}_t^2$, $\theta_t = \theta_{t-1} - \eta \mathbf{g}_t / (\sqrt{\mathbf{s}_t} + \epsilon)$	har bir parametrga o'z qadami
Adam	Momentum + RMSProp + siljish tuzatmasi	amaliy standart
AdamW	Adam + ajratilgan weight decay	Transformer/CNN standarti

($\mathbf{g}^2$, $\sqrt{\cdot}$ va bo'lish — elementlar bo'yicha.) Momentum: doimiy $\mathbf{g}$ uchun $\mathbf{v}_t \rightarrow \mathbf{g}/(1 - \mu)$, ya'ni $\mu = 0,9$ da samarali qadam 10η ; tebranuvchi komponentlar esa bir-birini yo'qotadi.

Xossa 8.1 (Kvadratik funksiyada gradient tushish). $f(\theta) = \frac{1}{2} \sum_i \lambda_i \theta_i^2$, $\lambda_i > 0$ bo'lsa, SGD (to'liq gradient) uchun $\theta_{t,i} = (1 - \eta \lambda_i)^t \theta_{0,i}$. Yaqinlashish $\iff 0 < \eta < 2/\lambda_{\max}$; eng yaxshi tezlik $\eta = 2/(\lambda_{\max} + \lambda_{\min})$ da, ko'paytuvchi $\frac{\kappa-1}{\kappa+1}$, $\kappa = \lambda_{\max}/\lambda_{\min}$.

Isbot. $\partial f / \partial \theta_i = \lambda_i \theta_i$, demak $\theta_{t,i} = (1 - \eta \lambda_i) \theta_{t-1,i}$. Barcha i uchun $|1 - \eta \lambda_i| < 1 \iff \eta < 2/\lambda_{\max}$. $\max_i |1 - \eta \lambda_i|$ minimumi $1 - \eta \lambda_{\min} = \eta \lambda_{\max} - 1$ da erishiladi. $\square$

Masalan, $\lambda = (1; 25)$, $\eta = 0,07 < 0,08$: 10 qadamdan keyin ko'paytuvchilar $0,93^{10} \approx 0,484$ va $(-0,75)^{10} \approx 0,056$ — tik yo'nalishda tebranib yaqinlashadi, yassi yo'nalishda sekin. Shartlanganlik soni κ katta bo'lsa (“tor vodiy”), SGD sekin; Momentum va adaptiv usullar aynan shu muammoga javob.

Misol 8.2 (SGD va Momentum qo'lda). $f = \frac{1}{2}(\theta_1^2 + 25\theta_2^2)$, $\theta_0 = (1; 1)$, $\eta = 0,01$, $\mu = 0,9$, $\mathbf{v}_0 = \mathbf{0}$.

Yechim. $\nabla f = (\theta_1; 25\theta_2)$. 1-qadam (ikkalasida bir xil, chunki $\mathbf{v}_1 = \mathbf{g}_1$): $\mathbf{g}_1 = (1; 25)$, $\theta_1 = (0,99; 0,75)$. 2-qadam. SGD: $\mathbf{g}_2 = (0,99; 18,75)$, $\theta_2 = (0,9801; 0,5625)$, $f \approx 4,435$. Momentum: $\mathbf{v}_2 = 0,9(1; 25) + (0,99; 18,75) = (1,89; 41,25)$, $\theta_2 = (0,9711; 0,3375)$, $f \approx 1,895$. 100 qadamdan keyin: SGD $f \approx 0,0670$ ($\theta_1 = 0,99^{100} \approx 0,366$ — yassi yo'nalishda sekin), Momentum $f \approx 4,1 \cdot 10^{-6}$. Inersiya yassi yo'nalishdagi kichik, lekin doimiy gradientlarni to'playdi. $\triangleleft$

8.1. Adam va siljish tuzatmasi

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, & \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \\ \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t}, & \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t}, & \theta_t &= \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \end{aligned}$$

Standart: $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\epsilon = 10^{-8}$, $\eta = 10^{-3}$.

Teorema 8.3 (Siljish tuzatmasi). $\mathbf{m}_0 = \mathbf{0}$ va $\mathbb{E}[\mathbf{g}_k] = \mathbf{g}$ (barcha k) bo'lsa, $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1^t) \mathbf{g}$, demak $\mathbb{E}[\hat{\mathbf{m}}_t] = \mathbf{g}$. $\mathbf{v}_t$ uchun ham xuddi shunday.

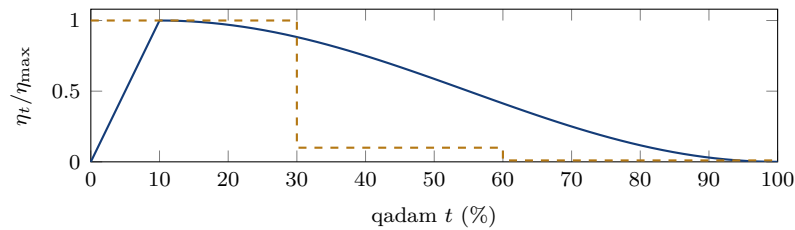
Isbot. Rekursiyani ochsak $\mathbf{m}_t = (1 - \beta_1) \sum_{k=1}^t \beta_1^{t-k} \mathbf{g}_k$, demak $\mathbb{E} \mathbf{m}_t = (1 - \beta_1) \mathbf{g} \sum_{j=0}^{t-1} \beta_1^j = (1 - \beta_1^t) \mathbf{g}$. $\square$

Misol 8.4 (Adam ning birinchi qadami). $\mathbf{g}_1 = (0,01; 100)$, $\eta = 10^{-3}$.

Yechim. $\mathbf{m}_1 = 0,1\mathbf{g}_1$, $\mathbf{v}_1 = 0,001\mathbf{g}_1^2$, $\hat{\mathbf{m}}_1 = \mathbf{g}_1$, $\hat{\mathbf{v}}_1 = \mathbf{g}_1^2$, qadam $\eta\mathbf{g}_1/(|\mathbf{g}_1| + \epsilon) \approx (0,001; 0,001)$ — gradient masshtabi 10^4 marta farq qilsa ham, qadam bir xil: Adam masshtabga (diagonal) invariant. Tuzatmasiz qadam $\eta \cdot 0,1g/\sqrt{0,001}|g| \approx 3,16\eta$ bo'lar edi. $\triangleleft$

AdamW. Adam ga L_2 jarima ($\mathbf{g} + \lambda\theta$) qo'shilsa, $\lambda\theta$ ham $\sqrt{\hat{\mathbf{v}}}$ ga bo'linadi — katta gradientli parametrlar kamroq “so'nadi”. AdamW (Loshchilov–Hutter) so'nishni ajratadi: $\theta_t = \theta_{t-1} - \eta(\hat{\mathbf{m}}_t/(\sqrt{\hat{\mathbf{v}}_t} + \epsilon) + \lambda\theta_{t-1})$. SGD da bu ikki yondashuv ekvivalent, adaptiv usullarda — yo'q.

Qadam jadvallari. Step decay (η har k epoxada γ marta kamayadi); cosine: $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos \frac{\pi t}{T})$; warmup: dastlabki T_w qadamda η chiziqli ravishda 0 dan $\eta_{\max}$ gacha oshiriladi (Adam ning $\hat{\mathbf{v}}$ bahosi hali shovqinli bo'lganda katta qadamlarning oldini oladi).



Ko'k: 10% warmup + cosine ($\eta_{\min} = 0$); sariq punktir: step decay ($\gamma = 0,1$).

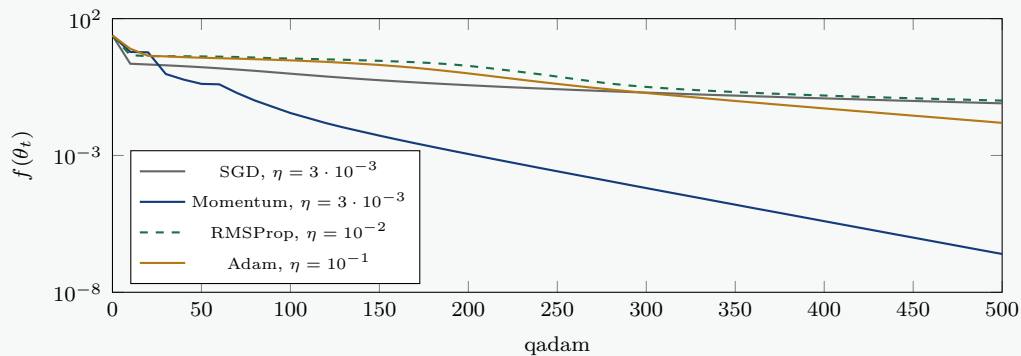
Amaliy talqin: o'qitish egri chiziqlarini “o'qish”

Yo'qotish nan yoki keskin o'sadi — η juda katta yoki gradient portlashi: η ni 3–10 marta kamaytir-ing, clipping qo'shing. *Yo'qotish deyarli o'zgaraydi* — η juda kichik, gradient so'ngan yoki xato (masalan, `zero_grad()` unutilgan, ikki marta softmax). *O'quv yo'qotishi kamayadi, validatsiya o'sadi* — overfitting: regularizatsiya (weight decay, dropout, augmentatsiya), erta to'xtatish. *Tebranuvchi egri* — paket kichik yoki η katta. Amaliy tekshiruv: avval bitta kichik paketni “yodlab olish” ($\text{yo'qotish} \approx 0$) — bu kod xatolarini tez aniqlaydi.

Amaliy talqin: Rosenbrock “vodiysi”

$f(\theta) = (1 - \theta_1)^2 + 100(\theta_2 - \theta_1^2)^2$, minimum (1,1), boshlanish (−1,2; 1), 500 qadam, har bir usul uchun $\eta \in \{10^{-4}, 3 \cdot 10^{-4}, \dots, 10^{-1}\}$ to'ridan eng yaxshisi:

Usul	Eng yaxshi η	$f(\theta_{500})$	Izoh
SGD	$3 \cdot 10^{-3}$	$8,0 \cdot 10^{-2}$	$\eta \geq 10^{-2}$ da uzoqlashadi (nan)
Momentum ($\mu = 0,9$)	$3 \cdot 10^{-3}$	$2,5 \cdot 10^{-7}$	eng yaxshi: vodiy bo'ylab tezlik to'playdi
RMSProp ($\rho = 0,9$)	10^{-2}	$1,0 \cdot 10^{-1}$	
Adam	10^{-1}	$1,6 \cdot 10^{-2}$	standart $\eta = 10^{-3}$ da $f \approx 4,24$



Hessian (1,1) da $\begin{pmatrix} 802 & -400 \\ -400 & 200 \end{pmatrix}$, $\kappa \approx 2508$ — juda tor vodiy. Saboq: universal eng yaxshi optimizator yo'q; η ni sozlash optimizator tanlashdan ko'ra muhimroq. Amaliy boshlanish: AdamW, $\eta = 10^{-3}$; tasvir klassifikatsiyasida ko'pincha SGD + Momentum ($\eta \approx 0,1$, jadval bilan).

9. Python'da amalga oshirish

0. Autograd qanday ishlaydi: 30 qatorlik skalyar reverse-mode. Har bir amal natija tugunini va uning mahalliy orqaga qoidasini (`_backward`) saqlaydi; `backward()` tugunlarni teskari topologik tartibda aylanib chiqadi.

```
import math
class Value: # skalyar reverse-mode autograd
    def __init__(self, data, parents=(), backward=lambda: None):
        self.data, self.grad, self._parents, self._backward = data, 0.0, parents, backward
    def __add__(self, o):
        out = Value(self.data + o.data, (self, o))
        def bw(): self.grad += out.grad; o.grad += self.grad
        out._backward = bw; return out
    def __mul__(self, o):
        out = Value(self.data * o.data, (self, o))
        def bw(): self.grad += o.data * out.grad; o.grad += self.data * out.grad
        out._backward = bw; return out
    def sigmoid(self):
        s = 1 / (1 + math.exp(-self.data)); out = Value(s, (self,))
        def bw(): self.grad += s * (1 - s) * out.grad
        out._backward = bw; return out
    def neglog(self): # -ln(p)
        out = Value(-math.log(self.data), (self,))
        def bw(): self.grad += -1 / self.data * out.grad
        out._backward = bw; return out
    def backward(self):
        order, seen = [], set()
        def topo(v): # topologik tartib
            if v not in seen:
                seen.add(v); [topo(p) for p in v._parents]; order.append(v)
        topo(self); self.grad = 1.0
        for v in reversed(order): v._backward()

w, x, b = Value(2.0), Value(1.0), Value(-1.0)
L = (w * x + b).sigmoid().neglog() # y = 1
L.backward()
print(round(L.data, 4), round(w.grad, 4), round(x.grad, 4), round(b.grad, 4))
# 0.3133 -0.2689 -0.5379 -0.2689 (grafdagi raqamli misol bilan bir xil)
```

PyTorch xuddi shu g'oyani tensorlar uchun amalga oshiradi: har bir tensor `grad_fn` orqali grafga bog'langan, `loss.backward()` esa grafni teskari aylanib chiqib, barg tensorlarning `.grad` maydoniga gradientlarni *qo'shadi*.

1. Umumiy MLP uchun backpropagation va gradient tekshiruvi (NumPy).

```
import numpy as np
rng = np.random.default_rng(0)
n, dims = 8, [4, 6, 5, 3] # 4 -> 6 -> 5 -> 3 (3 sinf)
X = rng.normal(size=(n, dims[0])); y = rng.integers(0, 3, n)
P = [(rng.normal(0, np.sqrt(2/a)), (a, b)), np.zeros(b)] for a, b in zip(dims, dims[1:])

def loss_and_grads(P, X, y, act=np.tanh, dact=lambda Z: 1 - np.tanh(Z)**2):
    A, Zs = [X], []
    for i, (W, b) in enumerate(P): # oldinga yurish
        Z = A[-1] @ W + b; Zs.append(Z)
        A.append(act(Z) if i < len(P) - 1 else Z)
    Z = A[-1] - A[-1].max(1, keepdims=True) # barqaror softmax
    logp = Z - np.log(np.exp(Z).sum(1, keepdims=True))
    L = -logp[np.arange(len(y)), y].mean()
    delta = np.exp(logp); delta[np.arange(len(y)), y] -= 1; delta /= len(y)
    grads = []
    for i in range(len(P) - 1, -1, -1): # orqaga yurish
        W, _ = P[i]
        grads.insert(0, (A[i].T @ delta, delta.sum(0)))
        if i > 0: delta = (delta @ W.T) * dact(Zs[i-1])
```

```

    return L, grads

L, G = loss_and_grads(P, X, y)
W = P[0][0]; num = np.zeros_like(W); eps = 1e-6 # sonli gradient (W1)
for idx in np.ndindex(W.shape):
    old = W[idx]; W[idx] = old + eps; Lp, _ = loss_and_grads(P, X, y)
    W[idx] = old - eps; Lm, _ = loss_and_grads(P, X, y); W[idx] = old
    num[idx] = (Lp - Lm) / (2*eps)
rel = np.abs(num - G[0][0]).max() / np.abs(num).max()
print(f"L={L:.6f}, nisbiy xato={rel:.1e}") # L = 1.461887, nisbiy xato = 4.9e-10

```

2. PyTorch autograd bilan solishtirish (yuqoridagi P, X, y, G davomi).

```

import torch
T = [(torch.tensor(W, requires_grad=True), torch.tensor(b, requires_grad=True)) for W, b in P]
h = torch.tensor(X)
for i, (W, b) in enumerate(T):
    h = h @ W + b
    if i < len(T) - 1: h = torch.tanh(h)
Lt = torch.nn.functional.cross_entropy(h, torch.tensor(y)); Lt.backward()
print(abs(Lt.item() - L),
      max(abs(W.grad.numpy() - g).max() for (W, _), (g, _) in zip(T, G)))
# 0.0 5.551115123125783e-17

```

Qo'lda chiqarilgan formulalar va autograd mashina aniqligi darajasida mos.

3. Optimizatorlar (Rosenbrock tajribasi).

```

def grad(t): return np.array([-2*(1-t[0]) - 400*t[0]*(t[1]-t[0]**2), 200*(t[1]-t[0]**2)])
def f(t): return (1-t[0])**2 + 100*(t[1]-t[0]**2)**2
def run(method, lr, steps=500, mu=0.9, rho=0.9, b1=0.9, b2=0.999, eps=1e-8):
    t = np.array([-1.2, 1.0]); v = np.zeros(2); s = np.zeros(2); m = np.zeros(2)
    for k in range(1, steps+1):
        g = grad(t)
        if method == 'SGD': t = t - lr*g
        elif method == 'Momentum': v = mu*v + g; t = t - lr*v
        elif method == 'RMSProp': s = rho*s + (1-rho)*g**2; t = t - lr*g/(np.sqrt(s)+eps)
        elif method == 'Adam':
            m = b1*m + (1-b1)*g; s = b2*s + (1-b2)*g**2
            t = t - lr*(m/(1-b1**k))/(np.sqrt(s/(1-b2**k))+eps)
    return f(t)
print(run('SGD', 3e-3), run('Momentum', 3e-3), run('RMSProp', 1e-2), run('Adam', 1e-1))
# 0.0803311... 2.50657e-07 0.1007839... 0.0155435...

```

4. To'liq o'qitish sikli (PyTorch): "ikki yarim oy" masalasi.

```

import numpy as np, torch, torch.nn as nn
def moons(n, noise, rng): # "ikki yarim oy" to'plami
    t = rng.uniform(0, np.pi, n); k = rng.integers(0, 2, n)
    x = np.where(k == 0, np.cos(t), 1 - np.cos(t))
    y = np.where(k == 0, np.sin(t), 0.5 - np.sin(t))
    X = np.c_[x, y] + noise * rng.normal(size=(n, 2))
    return torch.tensor(X, dtype=torch.float32), torch.tensor(k)
rng = np.random.default_rng(0)
Xtr, ytr = moons(450, 0.2, rng); Xva, yva = moons(150, 0.2, rng)
opts = {'SGD': lambda p: torch.optim.SGD(p, lr=0.1),
        'Momentum': lambda p: torch.optim.SGD(p, lr=0.1, momentum=0.9),
        'RMSProp': lambda p: torch.optim.RMSprop(p, lr=0.01),
        'Adam': lambda p: torch.optim.Adam(p, lr=0.01)}
for name, make in opts.items():
    torch.manual_seed(0)
    model = nn.Sequential(nn.Linear(2, 32), nn.ReLU(), nn.Linear(32, 16), nn.ReLU(),
                          nn.Linear(16, 2))
    opt, lossf = make(model.parameters()), nn.CrossEntropyLoss()
    for epoch in range(60):
        perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 32):

```

```

idx = perm[i:i+32]
opt.zero_grad()
loss = lossf(model(Xtr[idx]), ytr[idx])
loss.backward()
opt.step()
with torch.no_grad():
    Ltr = lossf(model(Xtr), ytr).item()
    acc = (model(Xva).argmax(1) == yva).float().mean().item()
    print(f'{name:9s} train_{Ltr:.4f} val_{acc:.4f}')

```

Natija (PyTorch 2.x, CPU, 60 epoxa, paket 32):

	SGD ($\eta = 0,1$)	Momentum ($\eta = 0,1$)	RMSPProp ($\eta = 0,01$)	Adam ($\eta = 0,01$)
O'quv yo'qotishi	0,0720	0,0661	0,0563	0,0703
Validatsiya accuracy	0,9600	0,9600	0,9667	0,9667

Oson masalada to'g'ri sozlangan optimizatorlar deyarli bir xil natija beradi (150 ta validatsiya namunasida farq bitta namuna); Rosenbrock kabi yomon shartlangan masalada esa farq katta. Boshqa variantlar va qadam jadvali:

```

opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=60) # har epoxa: sched.step()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # backward() dan keyin

```

Qadam tartibi har doim: `zero_grad()` → `backward()` → (clipping) → `step()`.

10. Xulosa

1. Backpropagation = zanjir qoidasi + teskari topologik tartib; barcha P gradient ≈ 3 ta oldinga yurish narxida (sonli usulda $2P$).
2. Softmax + cross-entropy: $\delta^{(L)} = \frac{1}{P - Y}$; softmax siljishga invariant, hisob log-sum-exp bilan.
3. $\nabla W^{(i)} = A^{(i-1)T} \delta^{(i)}$, $\nabla b^{(i)} = \delta^{(i)T} \mathbf{1}$, $\delta^{(i-1)} = (\delta^{(i)} W^{(i)T}) \odot \varphi'(Z^{(i-1)})$.
4. Oldinga yurishdagi faollashtirishlar saqlanadi — o'qitish xotirasi $\sim n \sum_i d_i$.
5. Gradient tekshiruvi: markaziy ayirma, $\varepsilon \sim 10^{-6}$, silliq faollashtirish; autograd bilan solishtirish.
6. φ' va $\|W\|$ ko'paytmasi gradientni so'ndiradi yoki portlatadi.
7. SGD tor vodiya sekin (κ); Momentum inersiya, RMSPProp/Adam adaptiv qadam beradi; Adam siljish tuzatmasi $1/(1 - \beta^t)$; AdamW so'nishni ajratadi.

11. Nazorat savollari

1. Nega sonli differensiallash o'qitish uchun yaroqsiz?
2. Hisoblash grafi va adjoint $\bar{v}$ nima?
3. Forward va reverse mode farqi nimada?
4. Softmax ning siljishga invariantligini isbotlang.
5. Softmax Yakobi matritsasini yozing.
6. Nega $\partial \ell / \partial \mathbf{z} = \mathbf{p} - \mathbf{y}$?
7. Nega model oxiriga Softmax qo'yilmaydi?
8. Backpropagation ning uchta formulasini yozing.
9. ∇W ning o'lchami nega W niki bilan bir xil?
10. Orqaga yurish uchun nima saqlanadi va nega?
11. Gradient tekshiruvda ε qanday tanlanadi?
12. Nega ReLU bilan gradient tekshiruvi xato berishi mumkin?
13. Vanishing va exploding gradient sabablari nima?
14. Gradient clipping qanday ishlaydi?
15. Momentum tor vodiya nega yordam beradi?
16. Adam dagi siljish tuzatmasi nimaga kerak?
17. AdamW ning Adam + L_2 dan farqi nimada?
18. Warmup va cosine schedule nima?

12. Mashqlar (javoblari bilan)

A guruh.

1. $\mathbf{z} = (1, 2, 3)$ uchun softmax va $y = 3$ bo'lganda cross-entropy. *Javob:* $\mathbf{p} \approx (0,0900; 0,2447; 0,6652)$, $\ell \approx 0,4076$.
2. Xuddi shu holatda $\partial \ell / \partial \mathbf{z}$. *Javob:* $(0,0900; 0,2447; -0,3348)$; yig'indi 0.
3. $\delta^{(i)}$: 32×64 , $W^{(i)}$: 128×64 . $\nabla W^{(i)}$, $\delta^{(i-1)}$ o'lchamlari? *Javob:* 128×64 ; 32×128 .
4. $\mu = 0,9$ Momentum da doimiy $\mathbf{g}$ uchun $\mathbf{v}_t$ limiti. *Javob:* $10\mathbf{g}$.
5. Gradient clipping: $\mathbf{g} = (6, 8)$, $c = 5$. *Javob:* $\|\mathbf{g}\| = 10$, $\mathbf{g} \leftarrow (3, 4)$.

6. 784–256–10 tarmoq, $n = 64$: orqaga yurish uchun saqlanadigan faollashtirishlar soni $(X, Z^{(1)}, A^{(1)}, Z^{(2)})$.
Javob: $64(784 + 256 + 256 + 10) = 83\,584$.

B guruh.

7. 5.1-misolda $\mathbf{b}^{(1)} = (0, 0)$ bo'lsa, $\nabla W^{(1)}$ ni toping. *Javob:* $Z^{(1)} = (1, 2)$, $Z^{(2)} = (3, -1)$, $\mathbf{p} \approx (0,9820; 0,0180)$,
 $\delta^{(2)} \approx (-0,0180; 0,0180)$, $\delta^{(1)} \approx (0; -0,0360)$, $\nabla W^{(1)} \approx \begin{pmatrix} 0 & -0,0360 \\ 0 & -0,0719 \end{pmatrix}$.
8. Softmax Yakobi matritsasi musbat yarim aniqlangan ekanini isbotlang. *Ko'rsatma:* $\mathbf{v}^T \mathbf{J} \mathbf{v} = \sum p_i v_i^2 - (\sum p_i v_i)^2$ — dispersiya.
9. $\lambda = (1; 25)$ kvadratik funksiyada SGD uchun η ning ruxsat etilgan oralig'i va optimal qiymati. *Javob:*
 $0 < \eta < 0,08$; $\eta^* = 2/26 \approx 0,0769$, ko'paytuvchi $12/13 \approx 0,923$.
10. $\mathbf{v}_t$ uchun Adam siljish tuzatmasi formulasini isbotlang.
11. Nol boshlang'ich og'irliklar bilan yashirin neyronlar gradienti bir xil bo'lishini ko'rsating. *Ko'rsatma:* $W^{(1)} = 0$ da $A^{(1)}$ ning barcha ustunlari teng; $W^{(2)} = 0$ bo'lsa, $\delta^{(1)} = 0$.
12. Label smoothing: $\tilde{\mathbf{y}} = (1 - \alpha)\mathbf{y} + \alpha \mathbf{1}/K$ uchun $\partial \ell / \partial \mathbf{z}$. *Javob:* $\mathbf{p} - \tilde{\mathbf{y}}$ (chunki $\sum \tilde{y}_j = 1$).

C guruh.

13. $L + \frac{\lambda}{2} \sum_i \|W^{(i)}\|_F^2$ uchun backpropagation formulalari qanday o'zgaradi? *Javob:* faqat $\nabla W^{(i)} = A^{(i-1)T} \delta^{(i)} + \lambda W^{(i)}$.
14. $\mathbf{g} = \mathbf{0}$ holatda Adam + L_2 va AdamW qadamlarini solishtiring ($t = 1$). *Javob:* Adam + L_2 : $\approx \eta \text{sign}(\theta)$ (λ ga bog'liq emas!); AdamW: $\eta \lambda \theta$.
15. To'liq bog'langan qatlam uchun oldinga va orqaga yurish FLOP larini hisoblang. *Javob:* $2nd_{i-1}d_i$ va $4nd_{i-1}d_i$ (birinchi qatlamda δW^T keraksiz: $2nd_0d_1$).
16. $\|\delta^{(1)}\|$ uchun yuqori baho chiqaring (so'nish/portlash).
17. Rosenbrock funksiyasining gradienti va $(1, 1)$ dagi Hessianini hisoblang, κ ni toping. *Javob:* $\kappa \approx 2508$ (xos qiymatlar $\approx 0,399$ va $1001,6$).
18. Nesterov momentumini NumPy'da yozing va Rosenbrock da oddiy Momentum bilan solishtiring.

Adabiyotlar

- [1] Goodfellow I., Bengio Y., Courville A. *Deep Learning*. MIT Press, 2016 (6, 8-boblar).
- [2] Rumelhart D. E., Hinton G. E., Williams R. J. Learning representations by back-propagating errors. *Nature*, 323, 1986.
- [3] Baydin A. G. va boshq. Automatic differentiation in machine learning: a survey. *JMLR*, 18, 2018.
- [4] Kingma D. P., Ba J. Adam: a method for stochastic optimization. *ICLR*, 2015.
- [5] Loshchilov I., Hutter F. Decoupled weight decay regularization. *ICLR*, 2019.
- [6] Glorot X., Bengio Y. Understanding the difficulty of training deep feedforward neural networks. *AISTATS*, 2010.
- [7] Nocedal J., Wright S. *Numerical Optimization*. 2nd ed. Springer, 2006.